LEVEL 5: CI/CD

5.3 Automated Testing in Pipelines

Topic 3 of 12 | ~5 hours | Difficulty: Intermediate

1 What it is

Automated testing in a pipeline means that every change is checked by machines, on a clean runner, before a human has to trust it. Tests come in layers that trade **speed** against **realism**:

- **Unit tests** exercise one function or class in isolation, with no network or database. Milliseconds each.
- **Integration tests** exercise several components together with *real* dependencies — your API talking to a real PostgreSQL and Redis. Seconds each.
- **End-to-end (e2e) tests** drive the whole running system the way a user or client would (a browser, or HTTP calls against a deployed environment). Seconds to minutes each.

The **test pyramid** is the rule of thumb for how many of each you should have: many fast unit tests at the base, fewer integration tests in the middle, a small number of e2e tests at the top. A **service container** is how a CI job gets real dependencies cheaply: the platform starts a throwaway PostgreSQL/Redis container next to your job. A **flaky test** is one that sometimes passes and sometimes fails on the same code — the single biggest threat to a team's trust in CI.

2 Why it matters in DevOps

- **Tests are the pipeline's first gate:** without them, "deploy automatically" just means "break production automatically".
- **Speed:** an upside-down pyramid (mostly slow e2e tests) makes the pipeline take an hour and developers stop running it.
- **Environment parity:** unit tests with mocks pass while the real SQL query is wrong; integration tests against real PostgreSQL catch it before staging does.
- **Trust:** if 1 in 20 runs fails randomly, engineers re-run until green and real failures slip through ("cry wolf").

- **Cost:** a failing test found in CI costs minutes; the same bug in production costs an incident.

3 Real-world system example

Google. In *Software Engineering at Google* and on the Google Testing Blog, Google describes classifying tests by size (small, medium, large) and recommends a distribution that is heavily weighted to small tests — roughly 80% small, 15% medium and 5% large is quoted as a guideline, not a law. Google's blog post *"Flaky Tests at Google and How We Mitigate Them"* is a public account of flakiness at scale: it reports that a small percentage of test runs were flaky and that a considerably larger share of individual tests showed some flakiness at some point, and describes mitigations such as re-running failing tests, tracking flakiness per test, and quarantining unreliable tests so they do not block everybody.

GitHub Actions service containers. GitHub documents service containers (PostgreSQL and Redis examples are in its own docs) precisely so projects can run integration tests against real databases in CI without maintaining shared test servers.

4 Diagram

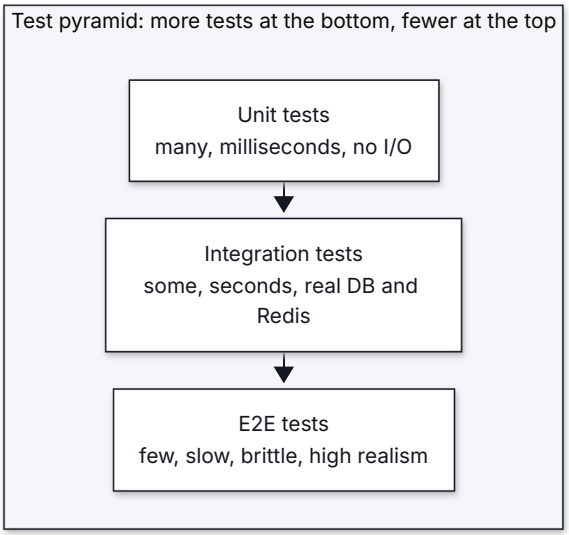


Figure 5.3-1

How an integration test job runs on a CI runner:

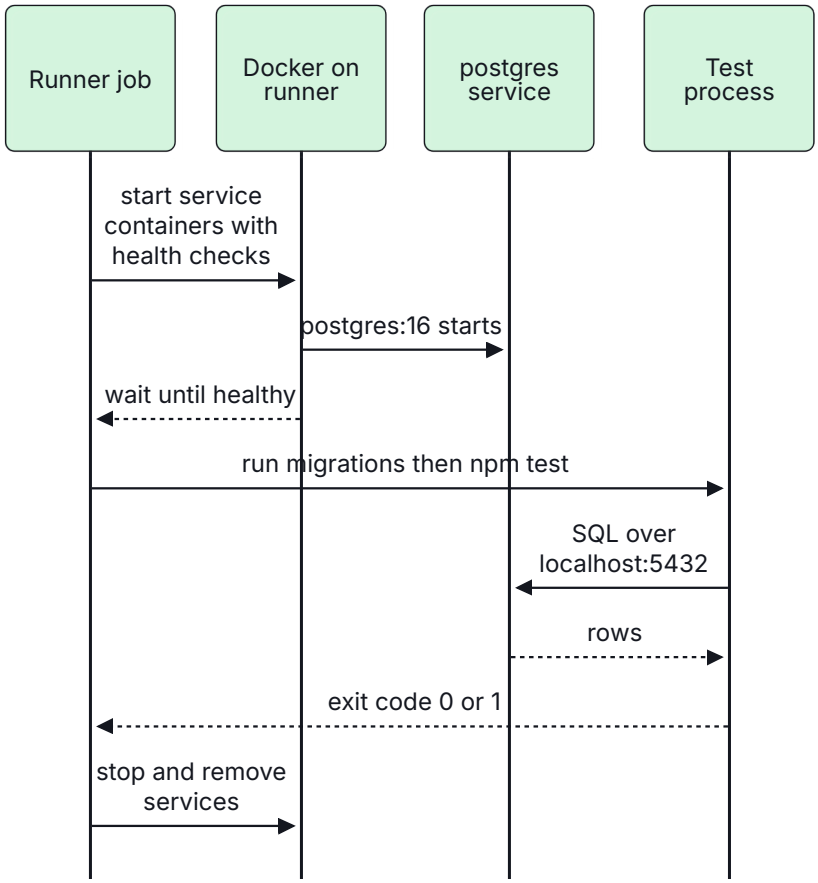


Figure 5.3-2

5 How it works

What belongs where

Layer	Scope	Dependencies	Speed target	Runs
Unit	one function or class	none (pure) or fakes	< 1 ms to 50 ms each; whole suite under 1-2 min	every push and PR
Integration	API + DB, API + cache, module boundaries	real PostgreSQL/Redis via service containers	whole suite a few minutes	every push and PR

Layer	Scope	Dependencies	Speed target	Runs
Contract	API shape between two services	stub or recorded provider	seconds	PR
E2E / smoke	the deployed system	a real environment	minutes	after deploy (staging), small set
Non-functional	load, security (DAST), accessibility	environment	long	nightly or before release

Sensible rules:

1. **Test the logic at the lowest layer that can see the bug.** A pricing rule is a unit test, not a browser test.
2. **Use real dependencies for integration, not mocks of the database.** Mocking the thing you most need to verify (SQL, migrations) proves nothing.
3. **Keep e2e small** — a handful of critical user journeys, plus a post-deploy **smoke test** (hit `/healthz`, fetch one resource).
4. **Make tests hermetic:** each test creates its own data, uses random ports, and does not depend on order or wall-clock time.
5. **Fail on coverage regressions sparingly:** a coverage gate (for example "lines >= 80%") prevents rot, but 100% coverage is a vanity target.

Service containers

In GitHub Actions, `services:` starts containers on the runner network before the steps. Health options make the job wait until the service is ready:

YAML

```yaml
services:
  postgres:
    image: postgres:16
    env:
      POSTGRES_PASSWORD: postgres
      POSTGRES_DB: shop_test
    ports:
      - 5432:5432
    options: >-
      --health-cmd "pg_isready -U postgres"
      --health-interval 5s
      --health-timeout 5s
      --health-retries 10
```

When the job runs directly on the runner VM (the default), reach the service at `localhost:<mapped port>`. When the job itself runs in a container (`container:`), reach it by service name (`postgres:5432`). Service containers exist for the life of the job and are destroyed with it, so every run starts with an empty database.

Flaky tests: causes and handling

Typical cause	Fix
Timing: `sleep(1)` then assert	wait for a condition with a timeout, not a fixed delay
Shared state / order dependence	isolate data per test; run in random order to expose it
External network calls	stub them, or use a local fake
Time and randomness	inject a clock and seed the RNG
Resource limits on CI	right-size runners; avoid port collisions with port 0
Race in the app itself	a real bug, the test is the messenger — fix the code

Policy that works: (1) **detect** flakiness by re-running a suspect test many times and by tracking pass rate per test; (2) **quarantine** a flaky test quickly (move it to a non-blocking job, open an issue with an owner and a deadline); (3) **fix or delete**; (4) allow *limited* automatic retries only as a visible, measured stopgap — silent retries hide bugs.

Making test output useful

- Emit a machine-readable report (JUnit XML, TAP) and upload it as an artifact §5.6 so failures can be inspected later.
- Print only what is needed; show the failing assertion and the diff.
- Shard long suites across parallel jobs (a matrix of `shard: [1,2,3,4]`) once a single job exceeds your time budget.

6 Step-by-step guide

Prerequisites: Node.js 22 (`node -v`), a GitHub repo. For the optional local part: Docker. Cost: free.

1. **Write a unit test.** Node has a built-in test runner, so no dependencies are needed.

```bash
mkdir -p src test
cat > src/price.js <<'EOF'
export function totalCents(items, taxRate = 0.2) {
  const subtotal = items.reduce((sum, i) => sum + i.priceCents * i.qty, 0);
  return Math.round(subtotal * (1 + taxRate));
```

```
}
EOF
cat > test/price.test.js <<'EOF'
import { test } from 'node:test';
import assert from 'node:assert/strict';
import { totalCents } from '../src/price.js';

test('adds 20 percent tax', () => {
  assert.equal(totalCents([{ priceCents: 1000, qty: 2 }]), 2400);
});

test('empty cart is free', () => {
  assert.equal(totalCents([]), 0);
});
EOF
echo '{"type":"module","scripts":{"test":"node --test"}}' > package.json
npm test
```

Expected (similar to): two lines `ok 1 - adds 20 percent tax` / `ok 2 - empty cart is free` and `# pass 2`, `# fail 0`.

2. **Add coverage and a gate.** Recent Node 22 releases include a coverage report and thresholds:

```bash
node --test --experimental-test-coverage --test-coverage-lines=90
echo "exit code: $?"
```

A table with `price.js` at 100% appears and the exit code is `0`. Set `--test-coverage-lines=101` to see the gate fail with exit code `1`.

3. **Write an integration test that needs PostgreSQL.** Run a local database first:

```bash
docker run -d --name pg-test -e POSTGRES_PASSWORD=postgres -e POSTGRES_DB=shop_test -p
  5432:5432 postgres:16
npm install pg
cat > test/db.test.js <<'EOF'
import { test, after } from 'node:test';
import assert from 'node:assert/strict';
import pg from 'pg';

const pool = new pg.Pool({
  connectionString: process.env.DATABASE_URL ??
  'postgres://postgres:postgres@localhost:5432/shop_test',
});
after(() => pool.end());

test('can create and read an order', async () => {
```

```
  await pool.query('CREATE TEMP TABLE orders (id serial PRIMARY KEY, total_cents int
  NOT NULL)');
  await pool.query('INSERT INTO orders (total_cents) VALUES ($1)', [2400]);
  const { rows } = await pool.query('SELECT total_cents FROM orders');
  assert.deepEqual(rows, [{ total_cents: 2400 }]);
});
EOF
npm test
```

Expected: three passing tests. Stop the database afterwards: `docker rm -f pg-test`.

4. **Run the same tests in CI with service containers.** Save as `.github/workflows/test.yml`:

```yaml
name: test
on:
  pull_request:
  push:
    branches: [main]

permissions:
  contents: read

jobs:
  test:
    runs-on: ubuntu-24.04
    timeout-minutes: 10
    services:
      postgres:
        image: postgres:16
        env:
          POSTGRES_PASSWORD: postgres
          POSTGRES_DB: shop_test
        ports:
          - 5432:5432
        options: >-
          --health-cmd "pg_isready -U postgres"
          --health-interval 5s
          --health-timeout 5s
          --health-retries 10
      redis:
        image: redis:7
        ports:
          - 6379:6379
        options: >-
          --health-cmd "redis-cli ping"
          --health-interval 5s
          --health-timeout 3s
          --health-retries 10
    env:
      DATABASE_URL: postgres://postgres:postgres@localhost:5432/shop_test
      REDIS_URL: redis://localhost:6379
```

```
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with:
          node-version: 22
          cache: npm
      - run: npm ci
      - name: Run tests with JUnit report
        run: node --test --test-reporter=spec --test-reporter-destination=stdout --
  test-reporter=junit --test-reporter-destination=junit.xml
      - uses: actions/upload-artifact@v4
        if: always()
        with:
          name: junit-report
          path: junit.xml
          retention-days: 7
```

In the run, expand **Initialize containers**: you see the PostgreSQL image pulled and a health status of `healthy` before the first step. The `junit-report` artifact appears on the run summary page. The password is a throwaway value for a database that only exists for this job.

5. **Add a post-deploy smoke test (the thin e2e layer).** After a deploy, a tiny script proves the deployed system answers:

```bash
cat > scripts/smoke.sh <<'EOF'
#!/usr/bin/env bash
set -euo pipefail
BASE_URL="${1:?usage: smoke.sh https://staging.example.com}"
curl -fsS --retry 5 --retry-delay 3 --connrefused "$BASE_URL/healthz" | jq -e
  '.status == "ok"'
curl -fsS "$BASE_URL/api/products" | jq -e 'type == "array"'
echo "smoke ok"
EOF
chmod +x scripts/smoke.sh
```

Run it later against a real environment, for example `scripts/smoke.sh http://localhost:3000`; output ends with `smoke ok`.

6. **Detect a flaky test.** Run the suite repeatedly and stop at the first failure:

```bash
for i in $(seq 1 30); do npm test --silent >/dev/null || { echo "FAILED on run $i";
  break; }; done; echo done
```

A deterministic suite prints just `done`. Add a deliberately flaky test (`assert.ok(Math.random() > 0.2)`) and re-run to see how quickly repetition exposes it.

7 Real-world scenario

(Illustrative.) The `shop-api` pipeline is red about one run in six on the `checkout` integration test. Engineers have started pressing "Re-run failed jobs" without reading the error. One day a real regression in the discount calculation is merged because "it was probably the flaky one", and customers are over-charged for four hours.

Post-incident the team measures: the same test passes or fails on identical commits. The test inserts an order and then `sleep(200)` before asserting that a Redis-backed cache has been invalidated; on a busy runner 200 ms is sometimes not enough. They replace the sleep with a poll-until-true helper (with a 5 s timeout), add a `shard` matrix to bring the suite under 6 minutes, and add a rule: any test failing twice in a week without a code change is quarantined within a day and assigned an owner. **Lesson:** a flaky test is not an annoyance, it is a security hole in your safety net.

8 Common mistakes and best practices

Don't

- Mock the database in the tests that exist to verify database behaviour.
- Share one long-lived test database between runs (state leaks, parallel runs collide).
- Use `sleep` to wait for asynchronous work.
- Normalise "just re-run it", or add silent automatic retries across the whole suite.
- Build an ice-cream cone: a huge e2e suite on top of a few unit tests.
- Put real credentials in test configuration; use disposable values for disposable services.

Do

- Run lint and unit tests first, then integration, then (post-deploy) smoke tests — fail fast.
- Use service containers with health checks so tests never start against a half-booted database.
- Upload test reports and coverage as artifacts; keep a flaky-test list with owners.
- Make each test create and clean its own data (transactions rolled back, temp tables, unique IDs).
- Keep the PR test run under ~10 minutes; shard or parallelise when it grows beyond that.

9 Tasks

- [ ] **T1 — Unit tests, locally** *(≈30 min, beginner)*
  - **Goal:** write and run unit tests with the built-in runner.

- **Steps:**
  1. Create `src/price.js` and `test/price.test.js` as in step 1.
  2. Add a test for a 0% tax rate and one for a quantity of 0.
  3. Run `npm test`.
- **Done when:** `npm test` reports your tests passing and exits with code 0.

☐ **T2 — Integration test with a real database** *(≈40 min, intermediate)*

- **Goal:** test code against real PostgreSQL.
- **Steps:**
  1. Start PostgreSQL in Docker (step 3).
  2. Write a test that creates a `products` table, inserts two rows and reads them back sorted by price.
  3. Stop the container and confirm the test now fails with a connection error.
- **Done when:** the test passes with the database up and fails clearly with it down.

☐ **T3 — Move it into CI with service containers** *(≈40 min, intermediate)*

- **Goal:** the same integration test runs on GitHub Actions.
- **Steps:**
  1. Add the workflow from step 4.
  2. Push a PR and open **Initialize containers** in the logs.
  3. Download the `junit-report` artifact.
- **Done when:** the run is green, shows both services healthy, and the artifact contains your test names.

☐ **T4 — Hunt a flaky test** *(≈40 min, intermediate)*

- **Goal:** detect, quarantine and fix flakiness.
- **Steps:**
  1. Add a test with `await new Promise(r ⇒ setTimeout(r, 10))` followed by an assertion on something that takes a random 0-30 ms.
  2. Run the loop from step 6 until it fails.
  3. Replace the sleep with a poll-until-true helper that has a timeout; re-run 30 times.
- **Done when:** the flaky version fails within 30 runs and the fixed version passes 30 of 30.

☐ **T5 — Add a coverage gate and break it** *(≈30 min, intermediate)*

- **Goal:** enforce a minimum coverage in CI.

- **Steps:**
  1. Add `--experimental-test-coverage --test-coverage-lines=80` to the CI test command.
  2. Add an untested function with several branches and push.
  3. Observe the failure, then add tests to fix it.
- **Done when:** CI is red because of the coverage threshold, then green after you add tests.

☐ **T6 — Prove it: choose the layer** *(≈30 min, advanced)*

- **Goal:** justify where each test lives.
- **Steps:**
  1. Without the guide, list five behaviours of `shop-api` (tax calculation, order insert, `/healthz` output, Redis cache expiry, checkout journey in a browser).
  2. Assign each to unit, integration, or e2e/smoke and give the reason and expected runtime.
  3. Explain to a teammate what your pipeline does when the checkout journey test fails twice in a week with no code change.
- **Done when:** you have a written table of five rows with reasoning and a stated quarantine policy.

10 Self-check questions

1. Why is a pyramid better than an "ice-cream cone" test suite?
2. Why use a real PostgreSQL service container instead of mocking the database client in integration tests?
3. How do you reach a service container from a step when the job runs directly on the runner? And when the job runs inside a container?
4. Give two common causes of flaky tests and the fix for each.
5. What is the purpose of a post-deploy smoke test, given that tests already passed in CI?

ANSWERS

1. Unit tests are fast, cheap and precise; e2e tests are slow, brittle and expensive. A pyramid gives quick, targeted feedback; a cone gives slow, vague feedback and flakiness.
2. A mock only proves the code calls the client as expected, not that the SQL, constraints, migrations and types work. A real database verifies behaviour.
3. Via `localhost:<mapped host port>` when on the runner; via the service name and container port (for example `postgres:5432`) when the job runs in a container.

4. Fixed sleeps → wait for a condition with a timeout. Shared state between tests → isolate data per test and randomise order. (Also: network calls → stub them; clock/randomness → inject them.)
5. CI tests the code in a test environment; the smoke test verifies the *deployed* system (configuration, secrets, networking, real dependencies) actually works, and it provides the signal for rollback.

11 Further reading

- *Software Engineering at Google* (Winters, Manshreck, Wright), chapters on testing — test sizes and the pyramid.
- Google Testing Blog: *Flaky Tests at Google and How We Mitigate Them*.
- Martin Fowler: *TestPyramid* and *Eradicating Non-Determinism in Tests* (martinfowler.com).
- GitHub Docs: *About service containers* (docs.github.com/actions).
- Node.js documentation: the `node:test` module and test runner (nodejs.org/api).
- See also Docker image builds in CI §5.4 and Dynamic testing with OWASP ZAP §11.13.